Use Case Document

for

PopulationReports Project

Contents

Use Case 1: Generate Country Report

Actor: User
Precondition: Application is running and connected to the world database.
Postcondition: Country report sorted by population is displayed or saved.
Basic Flow:

- User selects the country report option.
- Application queries the database for country data.
- Results are sorted and displayed.

Use Case 2: Generate City Report

Actor: User
Precondition: User has access to database; application is running.
Postcondition: City report sorted by population is displayed.
Basic Flow:

- User initiates the city report.
- Application retrieves city list from the database.
- Cities are sorted by population and displayed.

Use Case 3: Generate Capital City Report

Actor: User
Precondition: Capital-city mappings exist in the database.
Postcondition: Capital cities are displayed with population.
Basic Flow:

- User requests capital city data.
- Application runs a join query on country and city.
- Results are sorted and shown.

Use Case 4: Top N Populated Cities in a Country

Actor: User
Precondition: User inputs a valid country and number N.
Postcondition: Top N populated cities in the selected country are displayed.
Basic Flow:

- User enters country name and value of N.
- Application executes SQL query with LIMIT clause.
- Results are displayed in descending population order.

Use Case 5: Population Living in Cities vs Non-Cities (Country)

Actor: User
Precondition: City and country population data exists.
Postcondition: City vs. non-city population with percentages is shown.
Basic Flow:

- User selects a country.
- Application calculates total, urban, and rural population.
- Result is shown with percentage breakdown.

Use Case 6: World Population Report

Actor: User
Precondition: World database is populated.
Postcondition: Total global population is displayed.
Basic Flow:

- Application runs aggregation query.
- Result is fetched and displayed to the user.

Use Case 7: Region/Continent/Country/District/City Population Report

Actor: User
Precondition: User selects level and region exists in DB.
Postcondition: Total population for selected level is displayed.
Basic Flow:

- User selects region, continent, or country level.
- Application executes population query.
- Result is shown to the user.

Use Case 8: Language Population Report

Actor: User
Precondition: Language data is available in database.
Postcondition: Language speaker count and percentage of world population is shown.
Basic Flow:

- User selects languages (e.g., English, Chinese).
- Application queries and aggregates speaker counts.
- Results are displayed in descending order.

Use Case 9: Database Connection Test

Actor: Developer
Precondition: Database is accessible and credentials are valid.
Postcondition: Connection success or failure is asserted in logs.
Basic Flow:

- Developer runs JUnit test.
- App attempts connection to MySQL DB.
- Test verifies if connection is successful.

Use Case 10: Generate JAR with Maven

Actor: Developer
Precondition: Maven is installed and project compiles.
Postcondition: JAR file is generated inside `target/`.
Basic Flow:

- Developer runs `mvn clean install`.
- Project compiles without errors.
- JAR is generated in target folder.

Use Case 11: Create Dockerfile for Project

Actor: Developer
Precondition: JAR is built and Docker is installed.
Postcondition: Docker image is created from the Dockerfile.
Basic Flow:

- Developer writes Dockerfile using openjdk base.
- Adds built JAR to the image.
- Docker CLI is used to build the image.

Use Case 12: Run Tests on GitHub Actions

Actor: CI/CD Pipeline
Precondition: GitHub Actions is configured in repo.
Postcondition: Tests are run and results reported as badge.
Basic Flow:

- Code push triggers `maven.yml`.
- Dependencies are installed and tests executed.
- Test status is updated as a badge.

Use Case 13: Add Unit Tests

Actor: Developer
Precondition: Codebase contains testable logic.
Postcondition: Unit tests validate individual components.
Basic Flow:

- Developer writes JUnit test methods.
- Tests are added for critical modules.
- Results are reviewed and assertions validated.

Use Case 14: Add Integration Tests

Actor: Developer
Precondition: App is connected to a test DB.
Postcondition: Integration between modules and DB is validated.
Basic Flow:

- Developer writes integration tests with DB queries.
- Test data is validated against actual DB.
- Output is compared with expected results.

Use Case 15: Use Git and GitHub Effectively

Actor: Developer
Precondition: Git is installed and repo is initialized.
Postcondition: Codebase is versioned and collaboration enabled.
Basic Flow:

- Developer clones or creates GitHub repo.
- Uses branches: `master`, `develop`, `release`.
- Commits and pushes frequently.

Use Case 16: Create Kanban Board on Zube.io

Actor: Developer
Precondition: GitHub account is linked to Zube.io.
Postcondition: Project tasks are tracked using Kanban.
Basic Flow:

- Developer logs into Zube.io.
- Creates columns: Backlog, In Progress, Done.
- Adds tasks and links them to GitHub issues.

Use Case 17: Add README.md and Project Badges

Actor: Developer
Precondition: Repository exists on GitHub.
Postcondition: README shows project details and badges.
Basic Flow:

- Developer writes `README.md`.
- Adds badges: Build, Release, Code Coverage, License.
- Pushes to ‘master‘ branch.

Use Case 18: Define Code of Conduct

Actor: Maintainer
Precondition: Community standards are to be defined.
Postcondition: CODE_OF_CONDUCT.md file is available in repo.
Basic Flow:

- Maintainer adds `CODE_OF_CONDUCT.md`.
- Defines expected behavior and reporting process.

Use Case 19: Create Release from GitHub

Actor: Developer
Precondition: Project is complete and tagged.
Postcondition: GitHub Release is created with downloadable artifacts.
Basic Flow:

- Developer goes to GitHub → Releases.
- Clicks "Draft a new release."
- Selects tag, adds description, and publishes it.